Universidade Estadual de Campinas
Instituto de Computação

MC613 B

Relatório 1

Vending Machine

Autores:

Leonardo Ferreira	281289
Lucas Henrique Bertanha	276617
Rafael Tagliamento Santana	281359
Vinícius Maciel de Sousa	281391

Professor:
Casio Pacheco
Krebsl281289@dac.unicamp.br

Campinas – SP
2026

1 Planejamento

Conforme pedido, foram feitos dois diagramas na etapa inicial, o diagrama de blocos, visto na Figura 1, que representa um esquema de como pensamos em organizar o sistema; e o diagrama de estados, na Figura 2, que mostra como imaginamos o comportamento do sistema dado certas entradas e suas saídas.

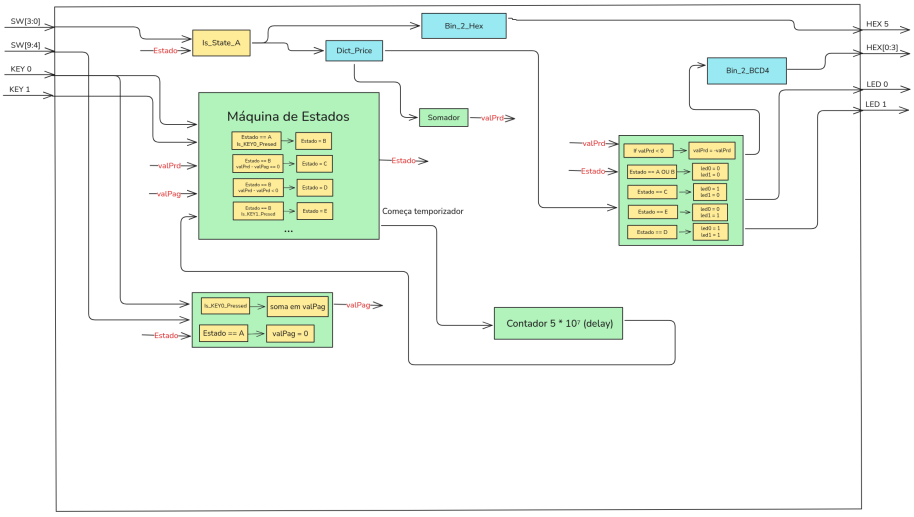

Figura 1: Diagrama de Blocos usado no planejamento do sistema.

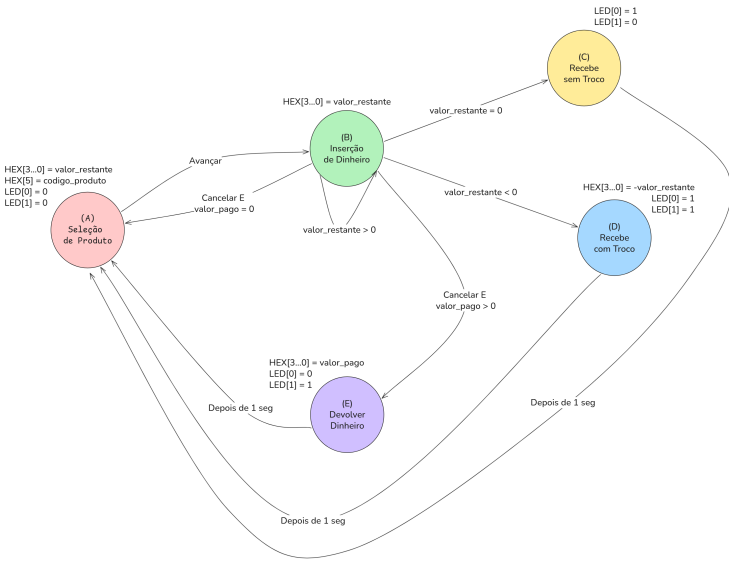

Figura 2: Diagrama de Estados usado no planejamento do sistema.

2 Estrutura do Código

O código do projeto foi feito seguindo as ideias dos diagramas da fase inicial, em que um arquivo central, a máquina de estados *states.v*, dita quais módulos devem ser executados de acordo com o contexto do sistema.

Além disso, outros módulos auxiliares foram necessários para realizar conversões e traduções, como por exemplo um módulo para transformar um binário de 11 dígitos em um binário BCD e outro que traduzia esse BCD para representar o número em um display de 7 segmentos. A funcionalidade de cada módulo do projeto está descrita abaixo:

- *bin11_to_bcd4*: módulo fornecido, converte uma entrada binária de 11 bits para 4 dígitos BCD;
- *bin2hex*: módulo fornecido, converte uma entrada binária de 4 bits, correspondente a um dígito, em uma saída binária de 7 bits correspondente aos leds do display de 7 segmentos que devem ser acesos para representar o dígito da entrada;
- *count_1s*: módulo que conta um tempo de aproximadamente 1 segundo, contando a partir da frequência da placa de $50MH_z$, fornecendo uma saída binária de verdadeiro ou falso, quando deu o tempo desejado;
- *displayer*: módulo que atualiza o valor monetário a ser mostrado pelos displays HEX[3...0] e o valor (ligado/desligado) a ser indicado pelos leds LEDR(0, 1);
- *payer*: módulo que converte uma entrada binária de 6 bits, correspondente ao valor da moeda, de acordo com o seu respectivo índice, em uma saída binária de 11 dígitos que é o valor pago anteriormente, somado ao valor que foi pago na operação;
- *pressure_button*: módulo responsável pelos cliques dos botões, recebendo a entrada do botão no momento atual e irá retornar uma entrada binária que representa um booleano, resultando verdadeiro, uma vez a cada click;
- *product2price*: módulo que converte uma entrada binária de 4 bits, correspondente ao índice do produto seleccionado, em uma saída binária de 11 bits, que é o preço daquele produto;
- *states*: módulo da máquina de estados, recebe o valor do produto, valor pago e as entradas dos botões e decide o próximo estado baseado nas condições atuais, conforme a Figura 2.

- *vending_ machine*: módulo com a responsabilidade de unir todos os outros (**top level**), recebendo os inputs da placa e passando os outputs para ela. Esse módulo instancia os outros módulos, tendo a responsabilidade de fazer todas as ligações entre as diferentes instâncias.

3 Implementação

```
1  module pressure_button
2  [...]
3    always @(posedge clk) begin
4      button_sync_0 <= button_in; // O botao sync ira receber a
       entrada do botao_0, 0 se pressionado ou 1 se nao esta
      pressionado
5      button_sync_1 <= button_sync_0; // sync 1 ira receber a
      entrada atual de sync 0
6    end
7
8    always @(posedge clk) begin
9      if(button_sync_1 == 1 && button_sync_0 == 0) begin //se o
       botao foi press ha um ciclo de clock
10       button_out <= 1;  //saida verdadeiro
11     end else begin
12       button_out <= 0;
13     end
14   end
15 endmodule
```

Listing 1: Código de Pressionar o botão

Módulo necessário para transformar um sinal contínuo vindo do pressionamento do botão em apenas um pulso, impedindo comportamentos inesperados. A linha 9 é que faz esse barramento, fazendo com que a "saída verdadeira"do botão, *button_ out*, seja alta apenas por um ciclo de clock.

```
1  module displayer
2  [...]
3  always @(posedge clk) begin // Usa clock para corrigir aviso
     de latch inferido
4    if (state == 3'b000 || state == 3'b001) begin // Estado A
     ou B, nao finalizou compra
5      ledr[0] = 1'b0;   // Apaga led 0
6      ledr[1] = 1'b0;   // Apaga led 1
7      prod_value_bin = prod_price - val_pago;
8    end else if (state == 3'b010) begin // Estado C, recebe sem
       troco
9      ledr[0] = 1'b1;   // Acende led 0
10     ledr[1] = 1'b0;   // Apaga led 1
```

```
11      prod_value_bin = 11'b0;
12    end else if (state == 3'b011) begin // Estado D, recebe com
        troco
13      ledr[0] = 1'b1;   // Acende led 0
14      ledr[1] = 1'b1;   // Acende led 1
15      prod_value_bin = val_pago - prod_price;
16    end else if (state == 3'b100) begin // Estado E, devolve
       dinheiro
17      ledr[0] = 1'b0;   // Apaga led 0
18      ledr[1] = 1'b1;   // Acende led 1
19      prod_value_bin = val_pago;
20    end else begin  // Estado impossivel
21      ledr[0] = 1'b0;   // Apaga led 0
22      ledr[0] = 1'b0;   // Apaga led 1
23      prod_value_bin = prod_price;
24    end
25  end
26  endmodule
```

Listing 2: Código responsável pelos leds e por atualizar o valor monetário a ser mostrado pelo display.

Módulo necessário para que nós tenhamos o valor monetário atualizado em tempo real para mostrar nos displays e os que os LEDs possam acender ou apagar do jeito correto. Os *ifs* das linhas 4, 8, 12 e 16 correspondem aos estados possíveis do sistema, e o da linha 20 é o adicional para impedir que ocorra um latch.

```
1  module states
2  [...]
3    always@(posedge clk) begin
4      next_state = state;
5      case(state)
6        3'b000: begin // A - selecao do produto
7          if(advance)
8            next_state = 3'b001;
9        end
10       3'b001: begin // B - insercao de dinheiro
11         if(cancel && val_pag == 0)
12           next_state = 3'b000;
13         else if(cancel)
14           next_state = 3'b100;
15         else if(val_prod == val_pag)
16           next_state = 3'b010;
17         else if(val_prod < val_pag)
18           next_state = 3'b011;
19       end
20       3'b010: begin // C - Recebe sem troco
21         if(prev_state != 3'b010)
```

```
22          reset_timer = 1;
23        else if(sec_pulse == 1) begin
24          reset_timer = 0;
25          next_state = 3'b000;
26        end else
27          reset_timer = 0;
28      end
29      3'b011: begin // D - Recebe com troco
30        if(prev_state != 3'b011)
31          reset_timer = 1;
32        else if(sec_pulse == 1) begin
33          reset_timer = 0;
34          next_state = 3'b000;
35        end else
36          reset_timer = 0;
37      end
38      3'b100: begin // E - Devolver dinheiro
39        if(prev_state != 3'b100)
40          reset_timer = 1;
41        else if(sec_pulse == 1) begin
42          reset_timer = 0;
43          next_state = 3'b000;
44        end else
45          reset_timer = 0;
46      end
47      default: begin
48        next_state = 3'b000;
49      end
50    endcase
51    prev_state = state;
52    state = next_state;
53  end
54 endmodule
```

Listing 3: Máquina de estados ...

Módulo que controla todo fluxo do sistema. Cada case, nas linhas 6, 10, 20, 29, 38, é um dos estados possíveis e dentro deles há diferentes subcasos em que valores são alterados. Nas linhas 8, 14, 16, 18, 25, 34 e 43 há atualizações do próximo estado do sistema, e na linha 47 há um case default novamenta para impedir latches.

```
1 module payer (
2 [...]
3
4 always @(posedge clk) begin
5     if (state == 3'b001) begin
6         if (advance) begin
7             case(coin)
8                 6'b100000: val_pago <= val_pago + 11'd200;
```

```
9                   6'b010000: val_pago <= val_pago + 11'd100;
10                  6'b001000: val_pago <= val_pago + 11'd50;
11                  6'b000100: val_pago <= val_pago + 11'd25;
12                  6'b000010: val_pago <= val_pago + 11'd10;
13                  6'b000001: val_pago <= val_pago + 11'd5;
14              default: val_pago <= val_pago;
15               endcase
16          end
17      end
18
19      else if (state == 3'b000) begin
20          val_pago <= 11'd0;
21      end
22  end
23
24  endmodule
```

Listing 4: O módulo de acrescentar o valor pago pelo usuário

Módulo necessário para atualizar quanto já foi pago pelo usuário. Cada linha de 8 até 13 corresponde a um valor de nota, que foi inserido como switches na placa. o *if* na linha 6 faz com que essa adição só funcione quando o botão for pressionado, e o default na linha 14 é novamente para impedir possíveis latches.

4 Validação

Abaixo, é possível verificar uma imagem do diagrama do circuito gerado no *software* que corresponde ao modelo desenvolvido no vigente projeto.

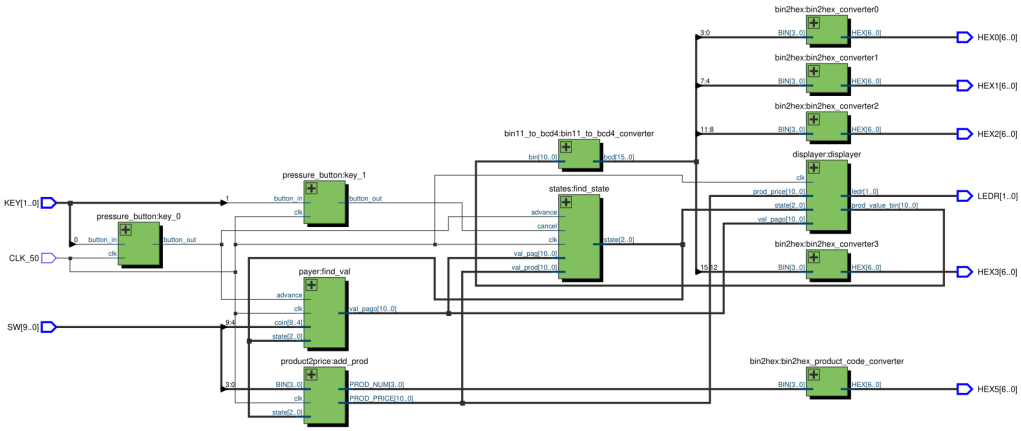

Figura 3: Diagrama do circuito completo

Dados os algoritmos desenvolvidos, promoveu-se a formulação de *test banches* que validassem a corretude dos algoritmos desenvolvidos. Esses testes foram desenvolvidos para os módulos de lógica complexa, deixando elementos mais intermediários e diretos para testes práticos.

Conversores:

Desenvolvemos inicialmente na Figura 4 os testes para o conversor de binário para BCD onde foram realizadas as conversões de 0 a 2047. A imagem mostra em *Wave* o caso binário 11010001101 que é convertido na saída para o valor BCD de 1677. No terminal também é possível visualizar as saídas para diferentes números com o formato "valor convertido = valor em BCD"

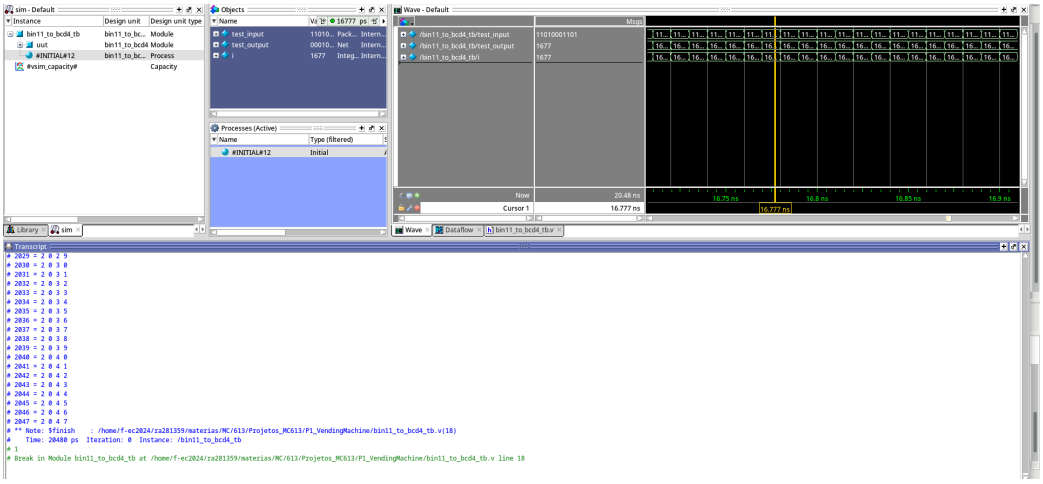

Figura 4: *Test bench* realizado para o conversor binário para BCD

Por conseguinte, na Figura 5, realizamos o test para o conversor de binário para hexadecimal. A visualização desses resultados é mais difícil. Tomando como exemplo o caso "6 = 0000010", é possível visualizar que o segmento 'b', representado pelo segmento superior da direita é o único elemento em 1, o que significa que ele está desligado, representando o número 6 num *display* de 7 segmentos.

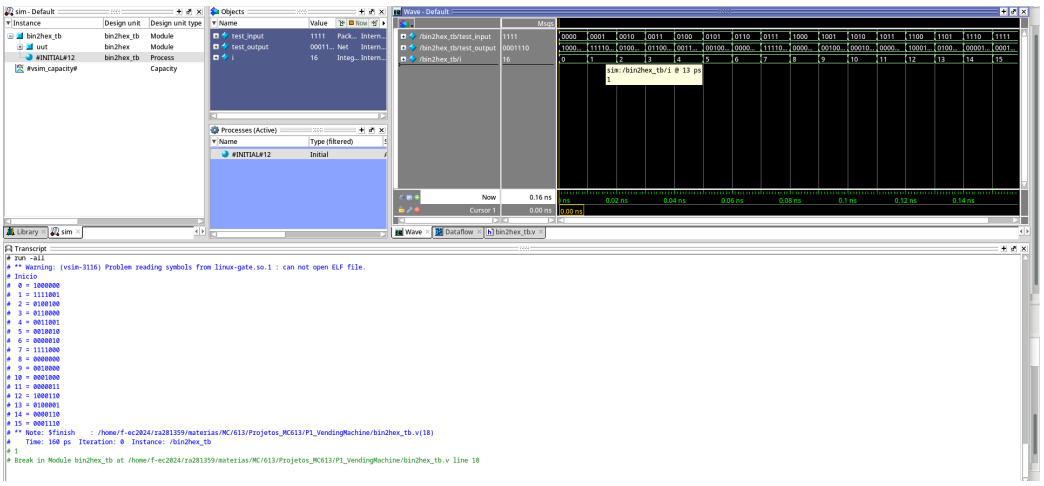

Figura 5: *Test bench* realizado para o conversor binário para hexadecimal

Máquina de estados:

Começamos agora uma sessão de testes para diferentes casos da máquina de estados. O maior foco é dado no valor de estado que será descrito acima de cada imagem. Vejamos:

Na Figura 6, desempenhamos um cancelamento com troco, onde saímos do estado A (000) para selecionar o produto, partimos para B (001) para pagar o produto, recebemos um valor pago menor que o de produto e ao cancelar partimos para E (100) onde cancelamos com o troco. O estado A retornará depois de cerca de 1 segundo.

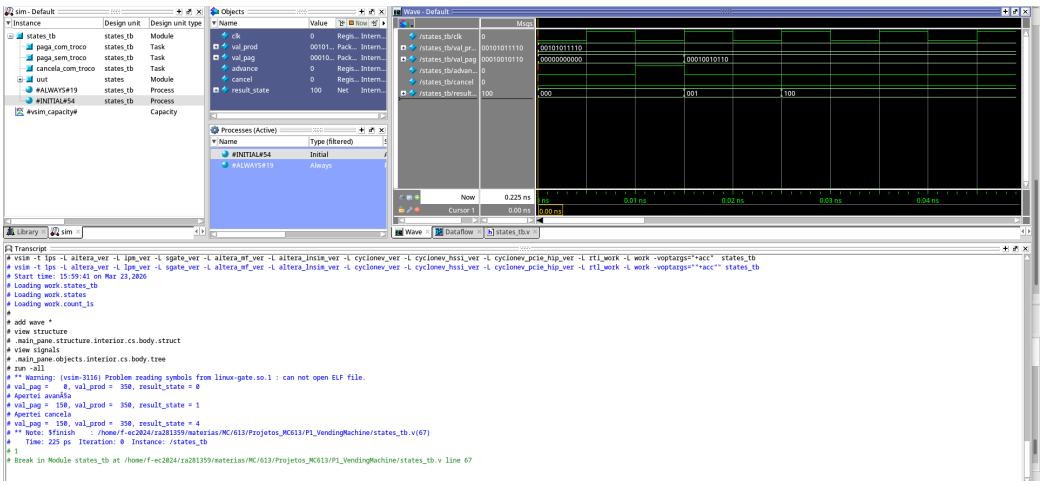

Figura 6: *Test bench* realizado para a máquina de estados realizando cancelamento com troco

Na Figura 7, desempenhamos um cancelamento com troco, onde saímos do estado A (000) para selecionar o produto, partimos para B (001) para pagar o produto, o valor pago continua nula e ao cancelar retornamos a A (000).

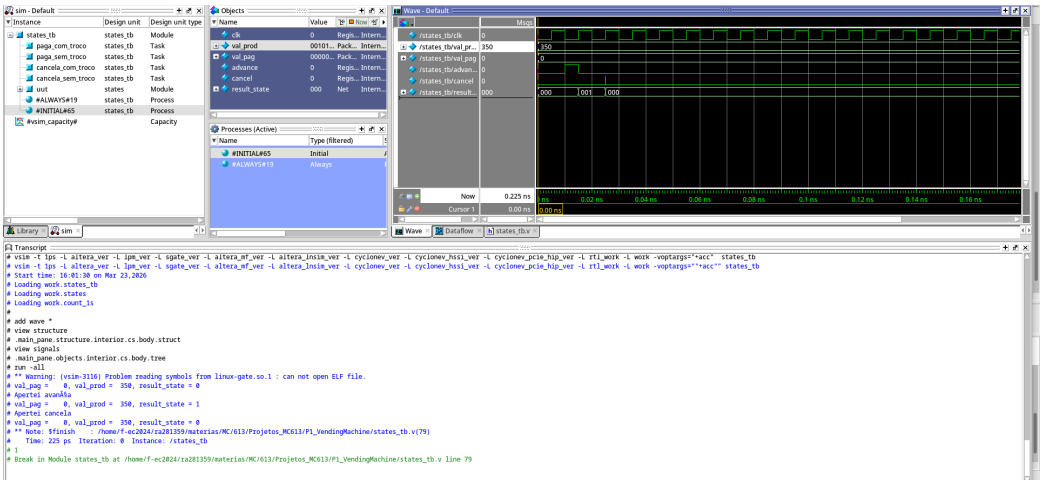

Figura 7: *Test bench* realizado para a máquina de estados realizando cancelamento sem troco

Na Figura 8, desempenhamos um cancelamento com troco, onde saímos do estado A (000) para selecionar o produto, partimos para B (001) para pagar o produto, recebemos um valor pago (400) maior que o de produto

(350) e assim partimos para D (011) onde pagamos e recebemos o troco. O estado A retornará depois de cerca de 1 segundo.

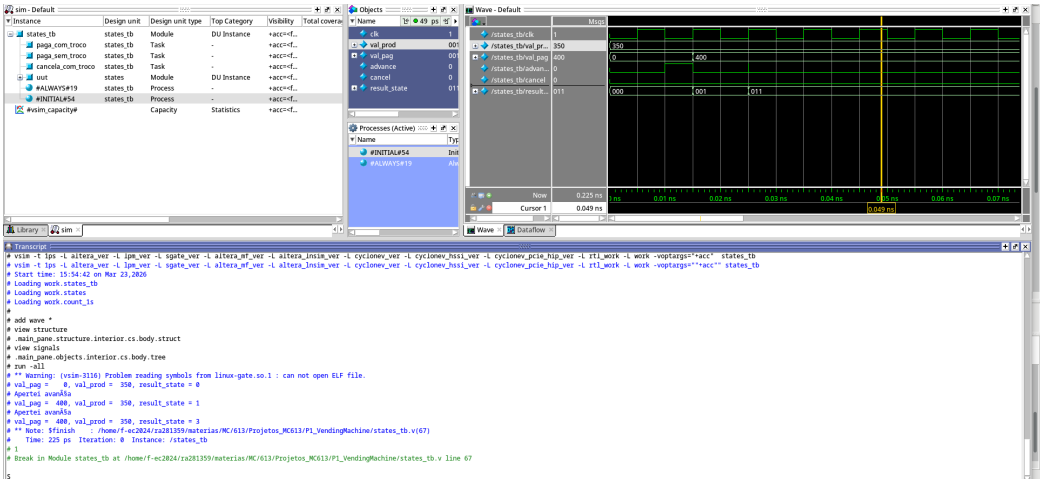

Figura 8: *Test bench* realizado para a máquina de estados realizando pagamento com troco

Na Figura 9, desempenhamos um cancelamento com troco, onde saímos do estado A (000) para selecionar o produto, partimos para B (001) para pagar o produto, recebemos um valor pago (350) igual ao valor de produto (350) e assim partimos para C (010) onde pagamos sem troco. O estado A retornará depois de cerca de 1 segundo.

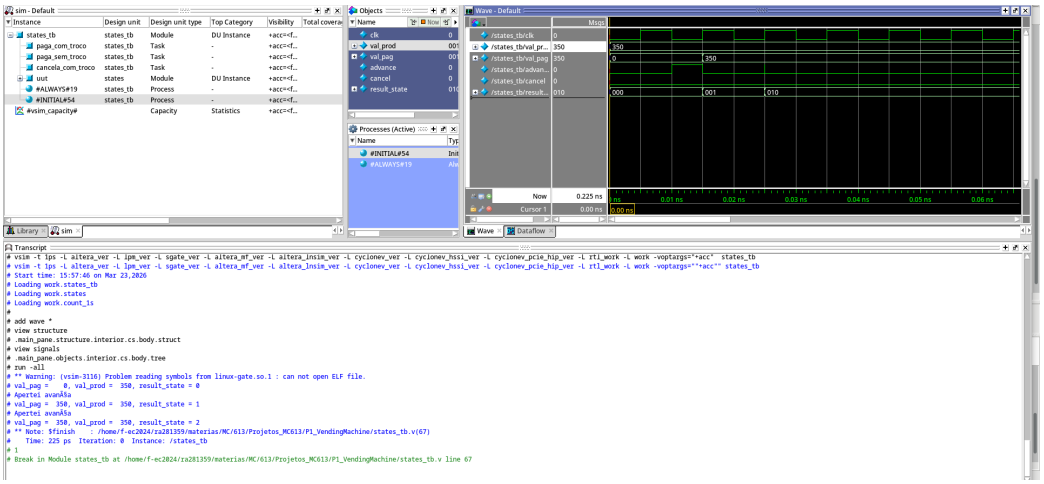

Figura 9: *Test bench* realizado para a máquina de estados realizando pagamento sem troco

Vending Machine

Segue-se para os testes realizados na máquina *top-level Vending Machine*

Na Figura 10, desempenhamos o cancelamento após já ter pago uma parte. Note que prod_value_bin é o valor que será mostrado na placa. Após escolher o produto 1, o botão avançar é pressionado, os switches de pagamento são ajustados para o pagamento de 200 e prod_value_bin vai para 100. O botão cancelar é pressionado e prod_value_bin mostra 200 (valor devolvido) como esperado. e ledr vai para 10 indicando a devolução do dinheiro.

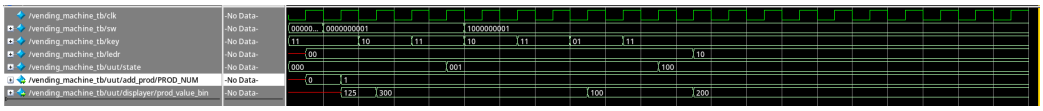

Figura 10: *Test bench* realizado para a *vending machine* para o caso de cancelamento com troco

Na Figura 11, desempenhamos o cancelamento sem ter pago nada. Note que prod_value_bin é o valor que será mostrado na placa. Após escolher o produto 1, o botão avançar é pressionado. O botão cancelar é pressionado e prod_value_bin continua mostrando 175 (preço do produto) como esperado. e ledr continua em 00.

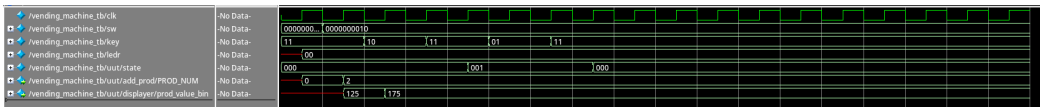

Figura 11: *Test bench* realizado para a *vending machine* para o caso de cancelamento sem troco

Na Figura 12, desempenhamos a tentativa de inserção de mais de uma moeda ao pagar. Após selecionar o produto 1 e clicar em avançar, sw é ajustado para 0110000001 e o botão avançar é pressionado. prod_value_bin não se altera como esperado

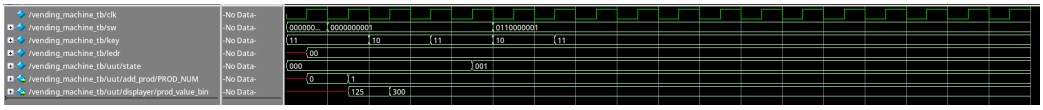

Figura 12: *Test bench* realizado para a *vending machine* para o caso em que o usuário coloca mais de uma moeda ao mesmo tempo

Na Figura 13, desempenhamos a tentativa de mudança do código de produto enquanto a compra já estava em andamento. Após selecionar o produto 1 e clicar em avançar, sw é ajustado para 0000000010. PROD_NUM não se altera como esperado.

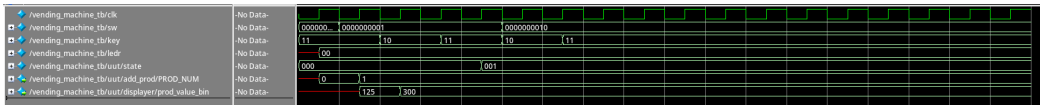

Figura 13: *Test bench* realizado para a *vending machine* de modo a testar a seleção de todos os produtos em estados onde não é permitido

Na Figura 14, é possível ver a seleção de todos os 16 produtos possíveis e como seus valores os acompanham após a descida de *clock* que se segue. O produto 0 gera prod_value_bin = 125, 1 gera prod_value_bin = 300 e assim por diante.

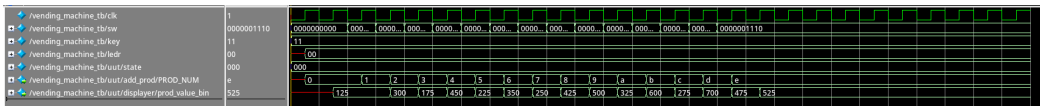

Figura 14: *Test bench* realizado para a *vending machine* de modo a testar a seleção de todos os produtos

Na Figura 15, desempenhamos o pagamento após já ter pago uma parte. Note que prod_value_bin é o valor que será mostrado na placa. Após escolher o produto, o botão avançar é pressionado, os *switches* de pagamento são ajustados para o pagamento até que prod_value_bin atinge o valor de 0. e ledr vai para 01 indicando confirmando a liberação do produto mas sem ativar a devolução de de dinheiro, já que o valor foi pago de forma exata.

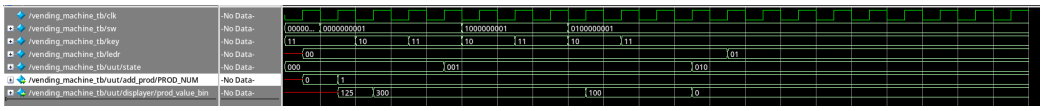

Figura 15: *Test bench* realizado para a *vending machine* para o caso de cancelamento sem troco

Na Figura 16, desempenhamos o pagamento após já ter pago uma parte. Note que prod_value_bin é o valor que será mostrado na placa. Após escolher o produto, o botão avançar é pressionado, os *switches* de pagamento são ajustados para o pagamento até que prod_value_bin atinge o valor de -90.

e ledr vai para 11 indicando os dois leds ligados confirmando a liberação do produto e a devolução de 90 que foi pago com sobra.

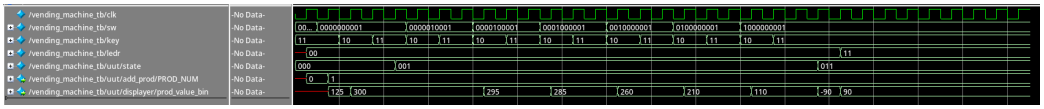

Figura 16: *Test bench* realizado para a *vending machine* para o caso de pagamento com troco

5 Análise de Execução

Na parte inicial de planejamento, a maior dificuldade foi entender como dividir o código em módulos e como conseguiríamos conectá-los entre si e com a máquina de estados. Isso porque era difícil ter uma ideia de como eles interagiriam sem uma ideia do sistema completo. A solução desse problema foi analisar cada estado um a um, e ir planejando os módulos necessários nessa ordem.

Na parte da codificação, a diferença de pensamento ao programar código que não é executado sequencialmente foi o mais complexo. Mas, além disso, houve dificuldade na implementação dos LEDs, feito no módulo *displayer*, cujo erro acabou sendo derivado do timer de 1 segundo, em *count_1s*, que foi diminuido para testes em simulação. Essa diminuição fez com que o display mudasse muito rápido na placa de volta para o estado inicial, o que foi interpretado como o código alterando estados incorretamente. Após a mudança desse timer, a placa funcionou corretamente.